Plasticity in Deep Reinforcement Learning

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Mohammad Mahdi Gheidi

universität freiburg

Master's Project

Submitted on **July 16, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Mohammad Mahdi Gheidi** (5973722)
Examined by **Prof. Dr. Joschka Bödecker**

Neurobotics Laboratory
Department of Computer Science (IIF)
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

Acknowledgements

I would like to thank my supervisor Dr. Shengchao Yan for his support during this work and letting me pursue what I find meaningful with this project.

And thanks to my family for their love and support throughout the way.

Abstract

Deep reinforcement learning has arisen to achieve remarkable success in domains such as game playing, robotics, recommendation systems, control, and large-scale decision making[Mnih et al., 2015, Silver et al., 2017]. However, despite these successes, modern deep reinforcement learning systems frequently suffer from a phenomenon known as plasticity loss.

Plasticity in general is the ability of a system to change how it perceives and processes the data and be able to adapt. In the context of human, Neuroplasticity is the brain's remarkable capacity to reorganize, grow, and rewire its neural connections throughout your lifetime. Rather than being a fixed organ, it continuously adapts to new experiences, learning processes, and environmental stimuli, or following an injury. [Puderbaugh and Emmady, 2023]. In the context of deep learning though, it refers to a neural network's ability to adapt its predictions when new information becomes available. As training progresses, networks often become increasingly difficult to modify, resulting in slower learning, reduced adaptability, and performance degradation in non-stationary environments[Lyle et al., 2023, Ash and Adams, 2020].

As we use DRL systems more and more by the day, we need to find the root causes preventing neural networks to benefit from plasticity while solving tasks in the reinforcement learning regime.

The objective of this project is to execute a literature study and investigate the mechanisms responsible for plasticity loss and understand how this phenomenon affects deep reinforcement learning systems. We would also reproduce the results based on the work [Lyle et al., 2023] and argue about the effect of geometrical landscape of functions and their effect on learning.

Contents

1 Motivation

Plasticity is becoming increasingly important as reinforcement learning systems are deployed in environments that continuously evolve.

Unlike traditional supervised learning, reinforcement learning operates under fundamentally non-stationary targets, especially when using value-based learning methods such as learning Q-values.

As the learning progresses during an RL task, policies and state values continuously change, data distributions evolve during training and target moves as learning objectives depend on previous updates, for example in case of temporal-difference learning methods.

As tasks become more complex, the ability of neural networks to remain adaptable becomes increasingly important. For example on longer horizon tasks and continouos action environments there is no other feasible solution than to use neural networks. In that case we need to make sure that this tool is a reliable function approximator to learn the dynamics of a changing environment.

Several recent works suggest that plasticity loss may be one of the major factors limiting the scalability of deep reinforcement learning. This has been noticed before in the deep learning regime, e.g. in the works purely done on supervised learning methods, we see that training the model on some part of the dataset at first and training it again on the second parts results in poor generalization [Ash and Adams, 2020]. Other works in the DRL have shown that, for example, as off-policy methods rely on using a replay buffer and use the data to train a model, will create the issue that the model overfits to early experiences in a way that harms future learning [Nikishin et al., 2022].

Understanding and preventing this phenomenon would lead to:

- More stable RL algorithms
- Faster adaptation
- Better continual learning
- Improved sample efficiency
- More robust autonomous agents

2 Fundamentals

The roots of this work can be traced to several related research areas.

2.1 Stability-Plasticity Dilemma

A long-standing challenge in neuroscience and machine learning is balancing stability and adaptability. Systems must preserve useful knowledge while remaining capable of learning new information.

Excessive stability leads to rigidity and inability to learn on new signals. Excessive plasticity leads to catasrophic forgetting.

2.2 Primacy Bias

[Nikishin et al., 2022] have already demonstrated that early experiences exert disproportionately large influence on neural network representations.

Networks become increasingly biased toward information learned early in training, reducing their ability to adapt later.

Also [Ash and Adams, 2020] showed that networks can become increasingly difficult to retrain after prolonged optimization.

Their observations provide evidence that optimization itself changes trainability.

2.3 Plasticity Loss

The most comprehensive recent study is: [Lyle et al., 2023] which studies “Understanding Plasticity Loss in Neural Networks”.

This paper provides strong evidence that plasticity loss is fundamentally related to changes in the geometry of the loss landscape rather than simply dead neurons, feature collapse or other previously mentioned improvements on the matter.

In the next section we cover the methodology used to carry out these experiments. We’ve based this project mainly on this paper.

3 Methodology

in order to be able to understand the issue happening underneath the training procedure of neural networks we need to define our methodology to do so.

3.1 Neural Network Architectures

To evaluate the effects of different neural network designs on plasticity, three commonly used architectures are considered throughout this project: a Multi-Layer Perceptron (MLP), a Convolutional Neural Network (CNN), and a Vision Transformer (ViT). These architectures were selected because they represent three fundamentally different approaches to feature extraction and representation learning, allowing the investigation of whether plasticity-related phenomena are architecture-dependent.

3.1.1 Multi-Layer Perceptron (MLP)

The MLP consists of two fully connected hidden layers. Unless otherwise specified, each hidden layer contains 512 neurons. For the network width scaling experiment presented later in this report, the hidden layer width is varied to investigate the relationship between model capacity and plasticity. In these experiments, a base width of 16 neurons is multiplied by scaling factors of 1, 2, 4, 8, 12, and 16, resulting in progressively wider networks.

3.1.2 Convolutional Neural Network (CNN)

The CNN architecture consists of two convolutional layers followed by two fully connected layers. The first convolutional layer employs 5×5 kernels, while the second uses 3×3 kernels. Both convolutional layers contain 64 feature maps. The extracted features are then processed by two fully connected layers, each containing 256 hidden units before producing the final output.

3.2 Datasets

We will be using the MNIST and CIFAR-10 datasets throughout the experiments. These datasets are among the most widely used benchmarks for image classification and provide two complementary levels of visual complexity. The MNIST dataset consists of 70,000 grayscale images of handwritten digits (60,000 training and 10,000 testing images), where each image has a resolution of 28 × 28 pixels and belongs to one of ten digit classes (0–9). Due to its simplicity, MNIST is commonly used for evaluating learning algorithms and analyzing optimization behaviour.

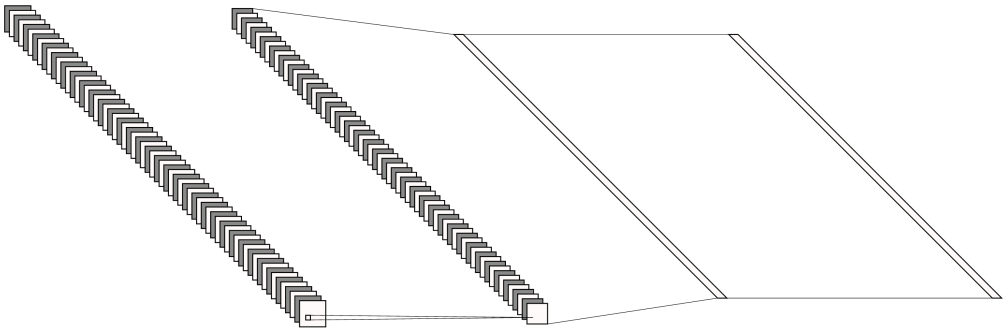

Figure 1: CNN on MNIST.

The CIFAR-10 dataset contains 60,000 colour images (50,000 training and 10,000 testing images), each with a resolution of 32 × 32 pixels and belonging to one of ten object categories, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Compared to MNIST, CIFAR-10 presents a substantially more challenging classification task because of its greater visual diversity, colour information, and more complex object features. Using both datasets enables the evaluation of neural network plasticity under learning tasks of varying complexity while maintaining a consistent ten-class classification setting.

Table 1: Datasets used throughout the experiments.

Dataset	Image Size	Channels	Classes	Training / Test
MNIST	28×28	1 (Grayscale)	10	60,000 / 10,000
CIFAR-10	32×32	3 (RGB)	10	50,000 / 10,000

3.3 Measuring Plasticity

The primary objective of this work is not to maximize reinforcement learning performance, but rather to understand and quantify the phenomenon of plasticity loss. Therefore, a precise operational definition of plasticity is required.

Following Lyle et al. [2023], plasticity is defined as the ability of a neural network to update its predictions in response to new learning signals. More formally, consider an optimization algorithm:

$$\mathcal{O} : (\theta, \ell) \mapsto \theta^*, \tag{1}$$

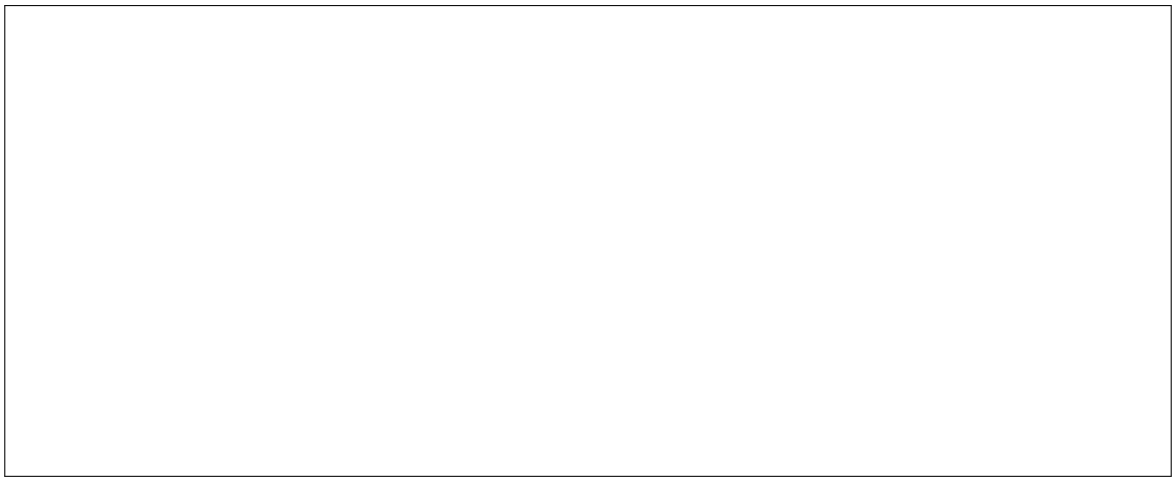

Figure 2: Overview of the three neural network architectures used throughout this work: (a) Multi-Layer Perceptron (MLP), (b) Convolutional Neural Network (CNN), and (c) Vision Transformer (ViT). The figure illustrates the high-level structure of each architecture and highlights their different approaches to feature extraction and representation learning.

which takes a parameter vector θ and a loss function ℓ and returns updated parameters θ^*. The resulting parameters need not correspond to a global optimum; for example, $\mathcal{O}$ may simply perform a fixed number of gradient descent steps.

To measure the flexibility of a network, a distribution over learning objectives is considered. In particular, **?** define a family of regression objectives

$$\ell_{f,X}(\theta) = \mathbb{E}_{x\sim X}\left[\left(f(\theta, x) - g\omega(x)\right)^2\right], \tag{2}$$

where g_ω defines a randomly generated target function.

To construct challenging prediction targets that require the network to modify its predictions in arbitrary directions, the target function is transformed according to

$$g(x) = a + \sin\left(10^5 f(x; \omega_0)\right), \tag{3}$$

where ω_0 is sampled from the same initialization distribution as the network parameters.

The offset parameter a is chosen to match the mean prediction of the network. This

prevents the probe task from favouring randomly initialized networks whose outputs tend to be centered around zero.

The resulting objective measures how easily a trained network can adapt its predictions to new targets. Networks capable of rapidly minimizing this objective are considered highly plastic, whereas networks that struggle to adapt are considered to have lost plasticity.

3.4 Loss Landscape Analysis

To better understand the optimization behaviour and plasticity of neural networks, this work analyzes the geometry of the loss landscape throughout training. In particular, two complementary quantities are investigated: the Hessian matrix of the loss function and the gradient covariance matrix. Together, these metrics provide insight into the curvature of the optimization landscape and the similarity of gradient directions across different training samples.

The Hessian matrix describes the second-order curvature of the loss function with respect to the network parameters. For a neural network with parameters θ and loss function $\ell(\theta)$, the Hessian is defined as

$$H_\ell(\theta) = \nabla^2_\theta \ell(\theta) \in \mathbb{R}^{d \times d}, \tag{4}$$

where $d = |\theta|$ denotes the total number of trainable parameters. Since the Hessian is generally a very large matrix, its eigenspectrum is analyzed instead,

$$\Lambda(H_\ell(\theta)) = (\lambda_1, \lambda_2, \ldots, \lambda_d), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d. \tag{5}$$

The eigenvalues quantify the curvature of the loss landscape along different directions in parameter space, while the corresponding eigenvectors indicate these principal directions of curvature. The largest eigenvalue is commonly used as a measure of the sharpness of the loss landscape, whereas the ratio between the largest and smallest eigenvalues (the condition number) provides an indication of the numerical conditioning of the optimization problem. Poor conditioning generally results in slower convergence and more difficult optimization.

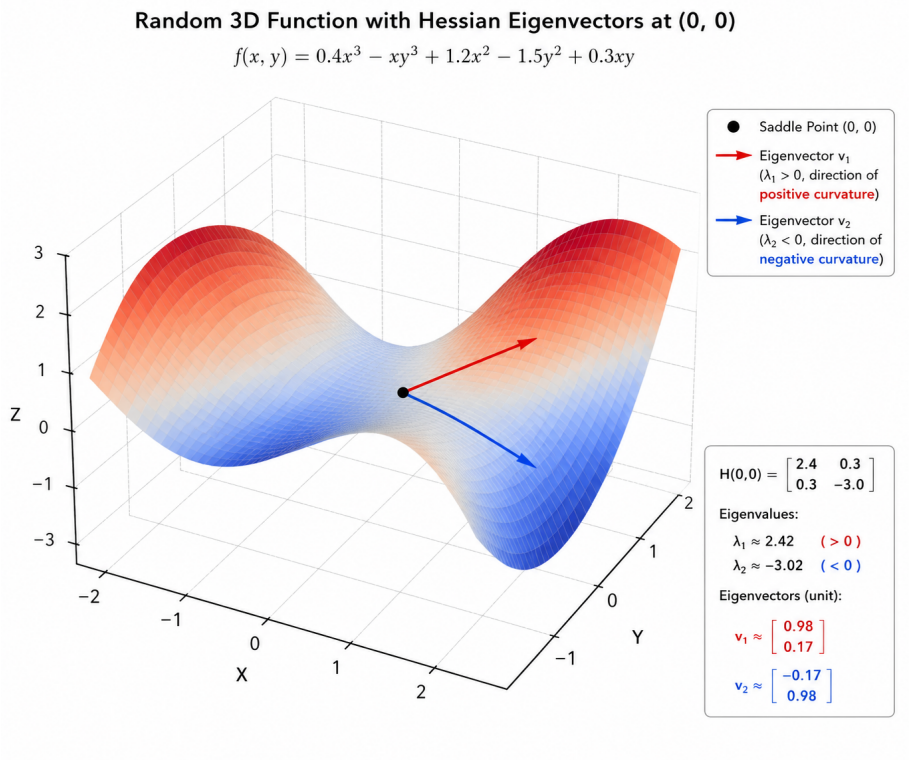

Figure 3: Illustration of a loss landscape together with the principal eigenvectors of the Hessian matrix. The eigenvectors indicate the directions of maximum and minimum curvature, while the corresponding eigenvalues quantify the curvature along those directions.

In addition to local curvature, this work also studies the similarity between gradients computed from different training samples. Let $x_1, \ldots, x_k$ denote a randomly sampled mini-batch of training examples. The gradient covariance matrix is computed as

$$C_k[i, j] = \frac{\langle \nabla_\theta \ell(\theta, x_i), \nabla_\theta \ell(\theta, x_j) \rangle}{\|\nabla_\theta \ell(\theta, x_i)\| \, \|\nabla_\theta \ell(\theta, x_j)\|}, \tag{6}$$

which corresponds to the cosine similarity between the gradients of every pair of training samples. Consequently, every entry of C_k lies within the interval $[-1, 1]$.

Positive values indicate that two samples produce similar parameter updates, suggesting cooperative learning and shared feature representations. Conversely, negative values imply that the corresponding gradients point in opposing directions, meaning that improving the performance on one sample may degrade the performance on the other. Such conflicting updates are commonly referred to as *gradient interference* and have been identified as an important factor affecting plasticity and continual learning.

Finally, the rank and overall structure of the gradient covariance matrix are examined throughout training. A low-rank covariance matrix indicates that many gradients lie within a small subspace of the parameter space, revealing shared optimization directions across the dataset. Visualizing this matrix as a heatmap further exposes clusters of highly correlated or anti-correlated gradients, providing additional insight into how the network organizes its internal representations as learning progresses.

3.5 Experimental Environments

To carry out the experiments in this project, we do not use a large-scale reinforcement learning benchmark, but rather toy environments to create controlled datasets in which plasticity can be measured reliably.

To achieve this, the methodology proposed by Lyle et al. [2023] converts standard supervised learning datasets into Markov Decision Processes (MDPs). This design provides a controlled setting where the effects of plasticity loss can be isolated from other challenges commonly encountered in reinforcement learning.

The experiments use the MNIST and CIFAR-10 datasets and consider three environment variants.

3.5.1 EasyMDP

EasyMDP corresponds to the true-label setting.

Each image is associated with its original dataset label. The objective of the agent is therefore identical to standard classification. The environment serves as a baseline in which the network learns meaningful representations from naturally structured targets.

3.5.2 HardMDP

HardMDP corresponds to the random-label setting.

Before training begins, a random label is assigned independently to each image. Consequently, images belonging to the same class may receive entirely different labels.

For example, one image of the digit 7 may be assigned label 5, while another image of the 7 may be assigned label 2.

This environment removes the underlying structure of the dataset and forces the network to memorize arbitrary associations. The setting is intentionally difficult and is used to investigate how plasticity evolves under extreme memorization demands.

3.5.3 Sparse-reward MDP

Sparse-reward MDP corresponds to the reinforcement learning setting used in the original paper.

In this environment, the agent receives a reward only when it selects the correct action in a designated goal state. In the experiments, state 9 is treated as the rewarding state.

The agent must therefore learn value estimates for all states and propagate reward information through the value function. Unlike EasyMDP and HardMDP, this environment introduces sparse rewards and temporal-difference learning, thereby capturing the essential characteristics of reinforcement learning.

3.6 Deep Q-Network Learning

A Deep Q-Network (DQN) [Mnih et al., 2015] is used throughout the experiments.

Although exact dynamic programming methods could be applied to represent the Q-values directly, such approaches would not allow investigation of the internal dynamics of neural network learning. Since the objective of this project is to study plasticity loss within neural networks, a function approximator is required.

The DQN learns action values according to the Bellman equation

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a'). \tag{7}$$

The network parameters are optimized using stochastic gradient descent on the temporal-difference loss

$$L(\theta) = \left(y - Q(s, a; \theta)\right)^2, \tag{8}$$

where

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{9}$$

and θ^- denotes the target network parameters.

Algorithm 1 summarizes the DQN training procedure.

Algorithm 1 Deep Q-Learning with Experience Replay

Initialize replay memory $\mathcal{D}$ with capacity N

Initialize action-value network $Q(s, a; \theta)$ with random parameters θ

Initialize target network $\hat{Q}(s, a; \theta^-)$ with $\theta^- \leftarrow \theta$

for episode $= 1, \ldots, M$ **do**

  Receive initial state s_1

  Preprocess state to obtain $\phi_1 = \phi(s_1)$

  for $t = 1, \ldots, T$ **do**

    With probability ϵ, select a random action a_t

    Otherwise select

$$a_t = \arg\max_a Q(\phi_t, a; \theta)$$

    Execute a_t, observe reward r_t and next state s_{t+1}

    Preprocess next state:

$$\phi_{t+1} = \phi(s_{t+1})$$

    Store transition

$$(\phi_t, a_t, r_t, \phi_{t+1})$$

  in replay memory $\mathcal{D}$

    Sample a random minibatch of transitions from $\mathcal{D}$

$$(\phi_j, a_j, r_j, \phi_{j+1})$$

    for all sampled transitions **do**

      if transition is terminal **then**

$$y_j = r_j$$

      else

$$y_j = r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-)$$

      end if

    end for

    Perform one gradient descent step on

$$(y_j - Q(\phi_j, a_j; \theta))^2$$

  with respect to θ

    if $t \bmod C = 0$ **then**

      Update target network:

$$\theta^- \leftarrow \theta$$

    end if

  end for

4 Results

Using the tools available at our disposal, we will try to do 6 experiments. There are 3 main questions we wish to gain more intuition on. First, we would like to see what would be the result of the networks losing plasticity. Using the first two experiments on trying to add jumps to the optimizers gradient using label change we show that both accuracy drops and many neurons die out. The second one shows that as the training progresses, the covariance between gradients increases and the eigenvalues of the Hessian matrix increase in size. As a result the optimization space will be ill-conditioned and harder to explore by the optimization algorithms. Eigenvalues are the non-turning vectors of a space, when applied an input on the space, those will be only affected in size, so they're a good sign to study the direction of optimization. Here we can see that they're gonna be bigger in value and make learning harder.

Second question we wish to answer, is to what properties would cause plasticity loss? in the 3rd experiment we investigate the effect of long-discussed weight norm, weight rank and other measures and show that there's no correlation. Then we move to our other experiment and try to measure the plasticity of trained networks after arbitrary learning time and see that plasticity drops as training proceeds.

At the end we put an effort to find solutions to alleviate this issue. Would increasing the size of the network be enough? We see that unlike other regimes, in which increasing the capacity will help like supervised learning, in the DRL world that we have non-stationary targets, simply increasing the size of the network has not that effect. Then using three architectures of simple MLPs, CNNs and Vision Transformers, we analyze the effect of few suggested methods and see that the most effective are layer normalization and categorical representations. Then we dive deeper into the effects of these two methods on the network and realize that their effect is to smoothen out the curvature of loss landscape.

4.1 Optimizer Instability Under Abrupt Label Changes

The first reproduced experiment corresponds to Figure 1 of Lyle et al. [2023]. The purpose of this experiment is to study how an adaptive optimizer behaves when the learning objective changes abruptly. In this setting, the network is first trained on MNIST images with randomly assigned labels. After the network has learned this mapping, the labels are randomly reassigned while the input images remain fixed.

This creates a sudden change in the gradient structure. The optimizer must now adapt

to a new objective that is inconsistent with the previously learned one. In the case of Adam, the first- and second-moment estimates no longer accurately describe the new gradient distribution. As a result, the effective update sizes can become unstable.

The reproduced experiment shows that this instability can cause a large fraction of hidden ReLU units to become inactive. Once these units die, the network loses effective representational capacity and its ability to learn the new task is strongly reduced. This provides an example of plasticity loss caused by optimizer dynamics under non-stationarity.

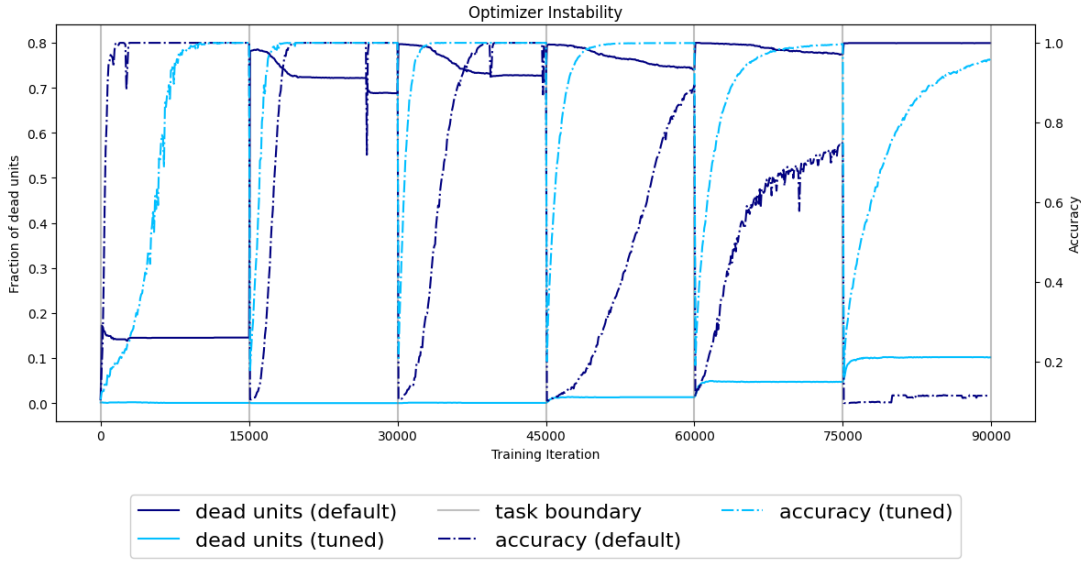

Figure 4: Abrupt label changes destabilize Adam and lead to a large fraction of inactive ReLU units.

4.2 Hessian Spectrum and Gradient Covariance

The second reproduced experiment corresponds to Figure 2 of Lyle et al. [2023]. The goal is to compare how the loss landscape evolves under gradient-based optimization versus random perturbations of the parameters. In both cases, the magnitude of the parameter change is controlled, but the direction of the update differs.

The Hessian spectrum is used to analyze the curvature of the probe loss landscape. Larger spectral outliers indicate sharper directions in parameter space and therefore potentially worse optimization conditioning. The reproduced Hessian analysis shows that gradient-based training tends to produce larger Hessian outliers than random Brownian perturbations. This suggests that training does not merely move the network away from initialization, but moves it in directions that make the local loss landscape harder to optimize.

In addition to the Hessian spectrum, gradient covariance matrices are computed to analyze interference between samples. Since the entries of the gradient covariance matrix are cosine similarities between normalized gradient vectors, their values must lie in the interval $[-1, 1]$. Therefore, in the reproduced plots we use the corrected color range $[-1, 1]$ rather than the wider range shown in the original visualization. This makes the interpretation clearer: positive values indicate aligned gradients, while negative values indicate gradient interference.

The reproduced gradient covariance results show that gradient-based optimization can create more negative interference between samples. This supports the hypothesis that plasticity loss is related not only to sharper curvature, but also to a less favorable gradient structure for adapting to new targets.

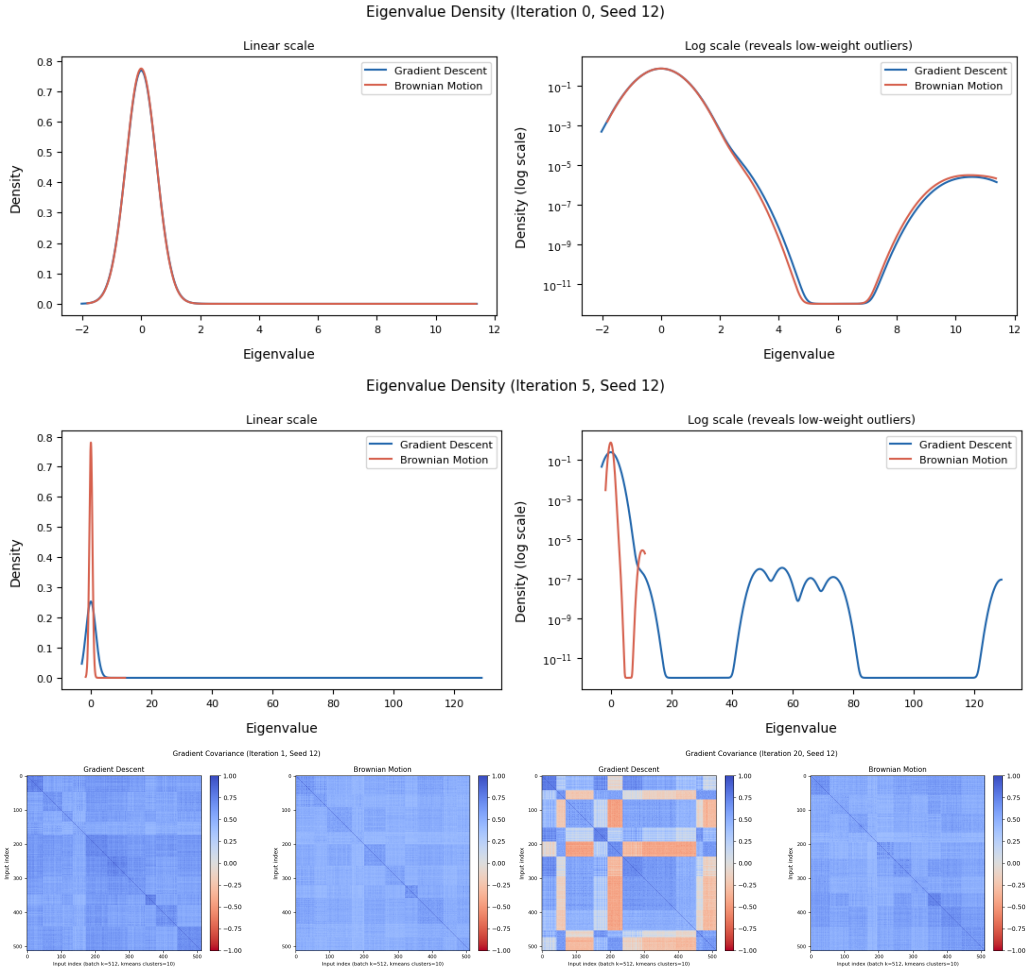

Figure 5: Hessian spectra and gradient covariance matrices are computed using the probe loss. For the gradient covariance heatmaps, the color scale is set to $[-1, 1]$ because the entries are cosine similarities.

4.3 Falsification of Simple Explanations

The third reproduced experiment corresponds to Figure 3 of Lyle et al. [2023]. This experiment investigates whether previously proposed quantities such as weight norm, weight rank, feature rank, or the number of inactive units can provide a causal explanation of plasticity loss.

The key idea is that a true causal mechanism should show a consistent relationship with plasticity across different environments. However, the reproduced results show that these quantities do not behave consistently. In some settings, the correlation with plasticity loss is positive, while in others it is negative or weak. Therefore, these quantities may be correlated with plasticity loss in particular cases, but they do not provide a general causal explanation.

Figure 6: Previously proposed explanations do not show a consistent relationship with plasticity loss across environments.

4.4 Learning Curve Evolution on Probe Tasks

The fourth reproduced experiment corresponds to Figure 4 of Lyle et al. [2023]. The purpose of this experiment is to understand whether plasticity loss corresponds to poor local minima or to slower optimization progress on new objectives.

The reproduced learning curves show that networks from later training checkpoints reduce the probe loss more slowly than networks near initialization. This suggests that plasticity loss is expressed primarily as slower adaptation, rather than simply convergence to a worse final plateau.

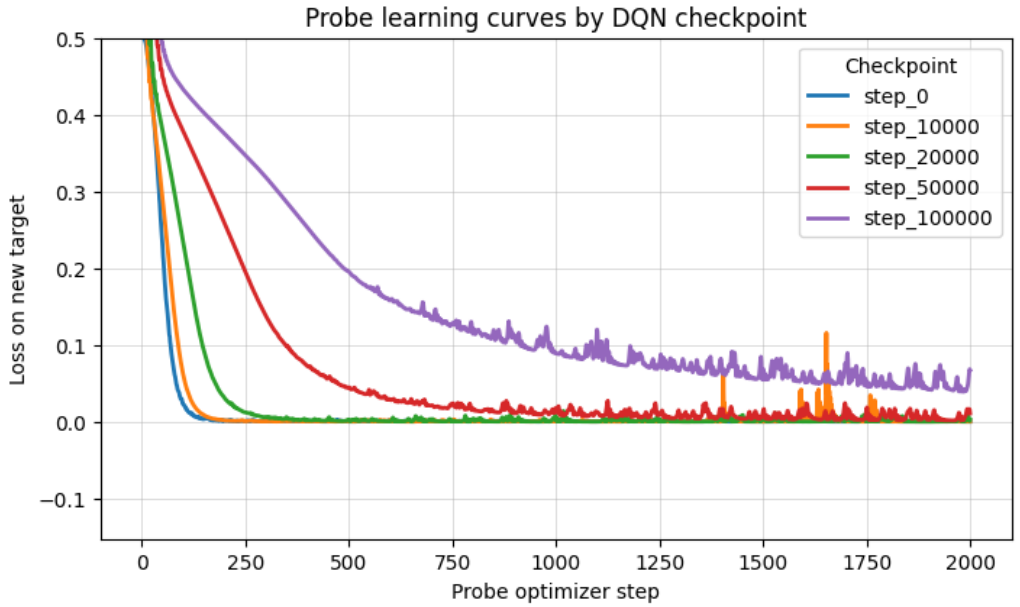

Figure 7: Later checkpoints show slower optimization on probe tasks, indicating reduced plasticity.

4.5 Effect of Network Width on Plasticity Loss

The fifth reproduced experiment corresponds to Figure 5 of Lyle et al. [2023]. This experiment studies whether increasing network width is sufficient to prevent plasticity loss.

The reproduced results indicate that wider networks generally suffer less plasticity loss than smaller networks. However, increasing width alone does not fully eliminate the problem. This suggests that plasticity loss is not merely a consequence of insufficient model capacity, but is also related to optimization dynamics and the geometry of the loss landscape.

4.6 Effect of Plasticity-Preserving Interventions

The sixth reproduced experiment corresponds to Figure 6 of Lyle et al. [2023]. This experiment compares several interventions designed to reduce plasticity loss, including layer normalization, reset-based methods, shrink-and-perturb, weight decay, spectral normalization, and categorical output representations.

The reproduced results support the conclusion that interventions which improve the smoothness and conditioning of the loss landscape are the most effective. In particular, layer normalization and categorical output representations show strong improvements compared to methods that only perturb or regularize the parameters. This supports the

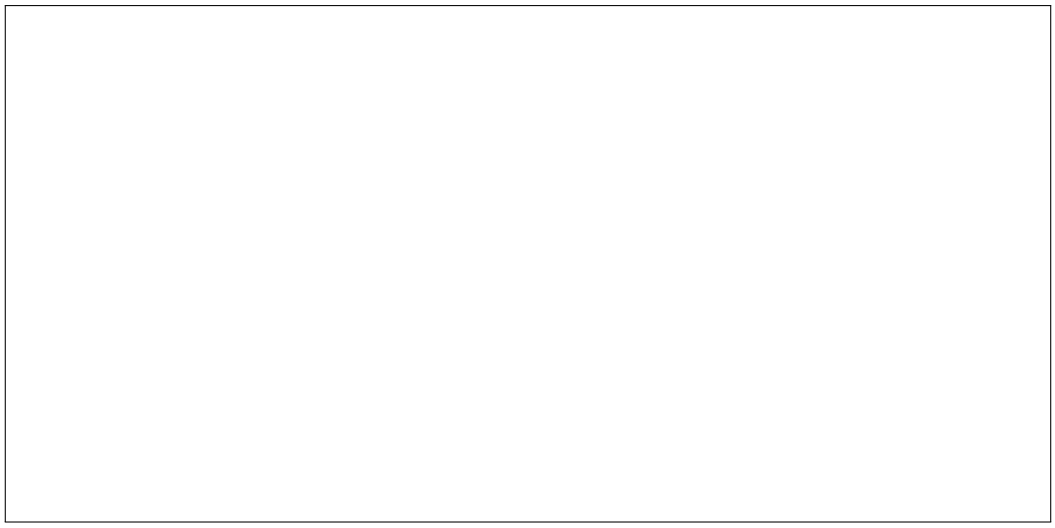

Figure 8: Increasing width reduces plasticity loss but does not fully remove it.

broader hypothesis that plasticity loss is closely related to loss landscape geometry.

Figure 9: Interventions that smooth or stabilize the loss landscape are most effective at preserving plasticity.

5 Conclusion

This project investigated the mechanisms behind plasticity loss in neural networks, with a particular focus on deep reinforcement learning. The main objective was to understand why networks that have already undergone training often become less capable of adapting to new learning signals. To address this question, we studied the recent literature on plasticity loss, capacity loss, warm-starting, and value-function learning, and reproduced several central experiments from [Lyle et al., 2023] to verify and get the understanding of what the authors have conveyed in this seminal work of theirs.

After reproducing these experiments in this project, we can support the view that plasticity loss is closely connected to the geometry of the loss landscape. In particular, the optimizer instability experiment showed that abrupt changes in the learning objective can significantly disturb the gradient structure and cause many hidden ReLU units to become inactive. This prevents the network from effectively using its representational capacity and can strongly reduce future learning ability. The Hessian and gradient covariance experiments further showed that training can move the network into regions of parameter space with sharper curvature and stronger gradient interference. These observations suggest that plasticity loss is not only a representational problem, but also an optimization problem.

Based on the reproduced experiments and the methods studied in [**?**], the most effective interventions were those that made loss curvature easier to optimize. In particular, methods such as layer normalization and categorical output representations were more successful than methods that only perturbed or regularized the parameters. This supports the hypothesis that preserving plasticity requires controlling the sharpness and conditioning of the loss landscape.

Layer normalization is especially relevant in this context because it changes the parameterization of the network in a way that can stabilize optimization. By normalizing intermediate activations, layer normalization can reduce sensitivity to scale changes and help prevent the network from entering extremely sharp regions of the loss landscape. In experiments of [Lyle et al., 2023], the addition of layer normalization reduced plasticity loss in several architectures. This suggests that normalization methods may help preserve plasticity by producing smoother and better-conditioned optimization dynamics.

The results are also consistent with the conclusions of [Farebrother et al., 2024], who

argue that value functions in deep reinforcement learning should not necessarily be trained as standard regression problems. In conventional value-based RL, the network is trained to regress directly to bootstrapped target values. This can be difficult because the targets are noisy, non-stationary, and unbounded. Regression objectives may therefore push the parameters into regions of the loss landscape from which later adaptation becomes difficult. In contrast, categorical methods such as two-hot encoding or HL-Gauss transform value learning into a classification-style problem. Instead of directly predicting a scalar value, the network predicts a distribution over a fixed support. This can make the learning problem better conditioned and more stable.

Further works can study methods to use to reduce the sharpness or smoothen the navigation of network in function space so that plasticity remains intact.

Bibliography

Jordan Ash and Ryan P Adams. On warm-starting neural network training. *Advances in neural information processing systems*, 33:3884–3894, 2020.

Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taïga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, et al. Stop regressing: Training value functions via classification for scalable deep rl. *arXiv preprint arXiv:2403.03950*, 2024.

Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *International Conference on Machine Learning*, pages 23190–23211. PMLR, 2023.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Evgenii Nikishin, Max Schwarzer, Pierluca D'Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *International conference on machine learning*, pages 16828–16847. PMLR, 2022.

Matt Puderbaugh and Prabhu D Emmady. Neuroplasticity. In *StatPearls [Internet]*. StatPearls Publishing, 2023.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.